

MEILENSTEIN 3 — IMPLEMENTIERUNG

GoTrip

Technische Dokumentation

MODUL

Mensch-Computer-Interaktion (HCI), SoSe 2026

HOCHSCHULE

h_da – Hochschule Darmstadt

TEAMMohamad Haj Ahmad · Wajdy Eleyan
Amal Najah · Eya Mathlouthi**STAND**

20. Juli 2026

INHALT

- | | |
|--|--------------------------|
| 1 Einleitung | 2 Umsetzung der Features |
| 3 Frontend-Entwicklung | 4 Backend-Entwicklung |
| 5 Responsivität & Benutzerfreundlichkeit | 6 Tests & Bug-Fixes |
| 7 Architektur-Entscheidungen | 8 Zusammenfassung |

1 / Einleitung

GoTrip ist eine mobile Web-App für die gemeinsame Gruppenreiseplanung. Gruppen koordinieren Verfügbarkeiten, Budgets und Interessen — die App berechnet daraus automatisch einen Score für jeden Reisevorschlag, basierend auf echten Klimadaten (Copernicus ERA5), Biodiversitätsdaten (GBIF) und NASA-Satellitendaten.

Dieses Dokument beschreibt die technische Umsetzung von Meilenstein 3: wie die in MS1 und MS2 erarbeiteten Spezifikationen in funktionierenden Code überführt wurden, welche Architekturentscheidungen getroffen wurden und welche Probleme dabei aufgetreten und gelöst wurden.

```
hybridScore = (kiScore / 100) × 0,4 + (starsAvg / 5) × 0,6
```

Kernformel aus MS2: KI-Score gewichtet 40 %, Gruppen-Sternebewertung 60 %.

2 / Umsetzung der Features

2.1 Abgleich mit den Spezifikationen

Alle in MS1/MS2 definierten Kernfeatures wurden vollständig implementiert:

FEATURE	UMSETZUNG	STATUS
Gruppenreise erstellen	<code>CreateTrip.tsx</code> + TripContext	Fertig
Mitglieder einladen	Einladungslink mit Base-57-Code	Fertig
Verfügbarkeit eintragen	AvailabilityCalendar (3 Zustände)	Fertig
Reisepreferenzen	Budget-Picker + 9 Interessen-Chips	Fertig
KI-Empfehlung	Score-Engine mit echten API-Daten	Fertig
Reiseziele bewerten	StarRating 0,5–5,0 (Halbsterne)	Fertig
Gruppenübersicht	Management-Seite + Schlussfolgerung	Fertig
Aktivitäten-Voting	Vorschläge + Thumbs-up	Fertig
Budget-Tracker	Ausgaben pro Mitglied erfassen	Fertig
Reiseplan	Itinerary mit Zeitslots	Fertig
Echtzeit-Sync	HTTP-Polling alle 6 Sekunden	Fertig
Mehrsprachigkeit	Deutsch, Englisch, Spanisch	Fertig

2.2 Dynamischer KI-Score

Im Vergleich zu MS2 wurde der Score-Mechanismus erweitert: Der Score wird nicht einmalig beim Hinzufügen gecacht, sondern **dynamisch bei jeder Anzeige** mit den aktuellen Gruppenpräferenzen neu berechnet

(`recomputeAnalysis()` in `scoreEngine.ts`).

```
// Interessenerfüllung – Beispiele aus scoreEngine.ts
case 'beach': return !coastal ? 8 : (warm ? 95 : 65)
case 'city':  return pop >= 500_000 ? 92 : pop >= 100_000 ? 72 : 18
case 'nature': return species > 1_000_000 ? 85 : 55
```

Frankfurt + Strand-Präferenz → Score 8, weil Frankfurt im Binnenland liegt (`coastal = false`). Basiert auf echten Küstendaten der Open-Meteo Marine API.

3 / Frontend-Entwicklung

3.1 Von Prototyp zu Code

Die in MS2 erstellten Prototypen dienten als direkte Vorlage. Jeder Screen entspricht einer React-Seite in `src/pages/`. Navigationsstruktur und visuelles Design wurden 1:1 übernommen.

3.2 Tech-Stack

SCHICHT	TECHNOLOGIE	VERSION
Frontend	React + TypeScript + Vite	React 19 , Vite 8
Styling	Tailwind CSS	v4
Routing	React Router	v7
State	React Context API	—
Persistenz lokal	localStorage	—
Persistenz server	PostgreSQL (Neon Cloud)	—
Backend	Express.js	—
Deployment	Render.com	—

3.3 Komponentenarchitektur

Die UI ist in drei Ebenen aufgeteilt:

- **Pages** (`src/pages/`) — vollständige Screens, je eine Route
- **Sheets** (`src/components/sheets/`) — Formular-Panels als Bottom-Sheets
- **Shared** (`src/components/shared/`) — PageHeader, BottomNav, Toast; global aktiv

3.4 State-Management

Auf externe Bibliotheken (Redux, Zustand) wurde bewusst verzichtet. React Context ist für den Umfang dieser App ausreichend:

- `AuthContext` — aktueller Nutzer (Name, ID)
- `TripContext` — alle Reisen, CRUD-Operationen
- `LanguageContext` — aktive Sprache + Übersetzungsfunktion `t(key)`

4 / Backend-Entwicklung

4.1 Express-Server

Der Backend-Server (`server/index.js`) erfüllt zwei Aufgaben:

1. **Gruppensynchronisation** — Reisedaten in PostgreSQL speichern, damit alle Mitglieder dieselben Daten sehen
2. **API-Proxy** — NASA Earthdata erfordert einen Bearer-Token, der nicht im Frontend liegen darf

4.2 Datenbank (Neon PostgreSQL)

Als Datenbank wurde **Neon** gewählt — ein serverloser PostgreSQL-Dienst mit kostenlosem Free-Tier. Die Tabelle `trips` speichert gebündelte Reisedaten als JSON.

```
// Sync-Endpunkt: PUT /api/trip
await pool.query(
  `INSERT INTO trips (code, bundle) VALUES ($1, $2)
   ON CONFLICT (code) DO UPDATE SET bundle = $2, updated_at = NOW()`,
  [code, bundle]
)
```

Sicherheit: `DATABASE_URL` und `EARTHDATA_TOKEN` liegen in `server/.env` — in `.gitignore` eingetragen, nie im Repository.

4.3 Echtzeit-Synchronisation

Da WebSockets im Free-Tier nicht dauerhaft offen gehalten werden können, wurde HTTP-Polling implementiert. Alle 6 Sekunden holt `pullTrip()` den aktuellen Stand aller Mitglieder.

```
Nutzer-Aktion
→ localStorage (sofort, offline-fähig)
→ scheduleSync() → POST /api/trip → Neon PostgreSQL
← pullTrip() alle 6 s ← andere Mitglieder
→ Toast-Benachrichtigung (auf allen Seiten sichtbar)
```

4.4 Externe APIs

API	VERWENDUNG
Open-Meteo Archive (ERA5)	Temperatur, Niederschlag, Sonnenstunden
Open-Meteo Marine	Küstenerkennung (Wellenhöhe)
Open-Meteo Geocoding	Koordinaten, Einwohnerzahl
GBIF Occurrence	Artenvielfalt (Biodiversitäts-Score)
NASA POWER	Sonneneinstrahlung, Luftfeuchte, Wind
NASA Earthdata	Satellitenbilder (optional, Token via Server)

5 / Responsivität & Benutzerfreundlichkeit

5.1 Mobile-First

Die App ist für Smartphones ausgelegt. Das Layout ist auf `max-width: 430px` begrenzt (iPhone-14-Breite) und zentriert sich auf größeren Bildschirmen. Alle Touch-Targets sind mindestens 44 × 44 px groß (Apple Human Interface Guidelines).

5.2 Getestete Geräte

- iOS Safari (iPhone 13 / 14) — primäre Zielplattform
- Android Chrome (Samsung Galaxy S22)
- Desktop Chrome, Firefox, Edge — voll funktionsfähig

5.3 Zugänglichkeit

- Alle interaktiven Elemente haben `aria-label`-Attribute
- StarRating hat `role="group"` und beschriftete Buttons pro Stern
- Farbkontraste erfüllen WCAG 2.1 AA
- Tastatur-Navigation möglich (`focus-visible`-Styles)

5.4 UX-Entscheidungen

- **Rote Badges** in der Bottom-Navigation — Mitglieder sehen sofort, wenn sich etwas geändert hat
 - **Toast-Benachrichtigungen** (2 Sekunden) — erscheinen global auf allen Seiten
 - **Loading-Skeleton** — animiertes Platzhalter-Layout während die KI-Analyse läuft
 - **Offline-fähig** — alle Daten zuerst in localStorage; App funktioniert ohne Server
-

6 / Tests & Bug-Fixes

6.1 Behobene Bugs

BUG	URSACHE	LÖSUNG
Datum geht beim ersten Versuch verloren	100 ms Debounce abgebrochen beim Unmount des WheelDatePickers	<code>useEffect</code> -Cleanup feuert <code>onSelect</code> mit <code>pendingVal</code> -Ref
Sternebewertung übernahm sich auf andere Ziele	SVG <code>clipPath</code> -IDs global identisch (<code>half-1</code> ... <code>half-5</code>)	<code>useId()</code> (React 18) erzeugt pro Instanz eindeutige IDs
Rote Badges erschienen unzuverlässig	<code>gotrip-sync</code> -Event feuerte nicht konsistent	4-Sekunden-Intervall als Fallback + count-basiertes Tracking
KI-Score ignorierte Präferenzänderungen	Score einmalig beim Hinzufügen gecacht	<code>recomputeAnalysis()</code> berechnet Score bei jeder Anzeige neu

6.2 Teststrategie

- **Mehrbenutzer-Test** — zwei Browser-Tabs; Änderungen synchronisieren sich nach max. 6 Sekunden
- **Offline-Test** — Server ausschalten; App zeigt Daten aus localStorage
- **Grenzwerte** — Reise ohne Mitglieder, ohne Präferenzen, Ziel mit Ladefehler
- **Cross-Browser** — Chrome, Firefox, Edge, Safari (iOS)

7 / Architektur-Entscheidungen

Warum localStorage + PostgreSQL?

localStorage ermöglicht sofortige, offline-fähige Reaktionen ohne Netzwerkverzögerung. PostgreSQL (Neon) dient ausschließlich zur Gruppensynchronisation. Dieses Muster ist robuster als eine reine Server-Lösung, weil die App auch bei Serverausfall vollständig nutzbar bleibt.

Warum regelbasierter Score statt echtem LLM?

Ein echter LLM-API-Call (OpenAI, Claude) kostet Geld pro Anfrage und ist für einen Hochschul-Prototypen nicht kostenfrei betreibbar. Die regelbasierte Engine liefert nachvollziehbare, deterministisch erklärbare Ergebnisse aus echten Messdaten — für ein Bewertungssystem transparenter als ein LLM-Output.

Warum kein Redux?

Für die Datenmenge und Komplexität dieser App ist React Context ausreichend. Redux hätte erheblichen Boilerplate ohne messbaren Nutzen erzeugt.

8 / Zusammenfassung

GoTrip erfüllt alle Anforderungen aus Meilenstein 3:

- | | |
|---|---|
| ✓ React 19 (über Minimalanforderung React 16) | ✓ Alle Kernfeatures aus MS1/MS2 implementiert |
| ✓ Mobile-first, responsive auf iOS und Android | ✓ Backend mit PostgreSQL-Datenbankanbindung |
| ✓ 6 externe APIs (Copernicus, GBIF, NASA, Open-Meteo) | ✓ Echtzeit-Sync zwischen Gruppenmitgliedern |
| ✓ Bugs identifiziert, dokumentiert und behoben | ✓ Deployment: gotrip-9id7.onrender.com |